Apex Test Class in Salesforce

How to build an almost perfect Apex test class in Salesforce

Guilherme Nunes Zwipp

Salesforce Developer
https://www.linkedin.com/in/guilherme-zwipp-salesforce/

Target Audience and Objective

Objective: Explain what Apex test classes in Salesforce are, their importance, and how to implement them effectively.

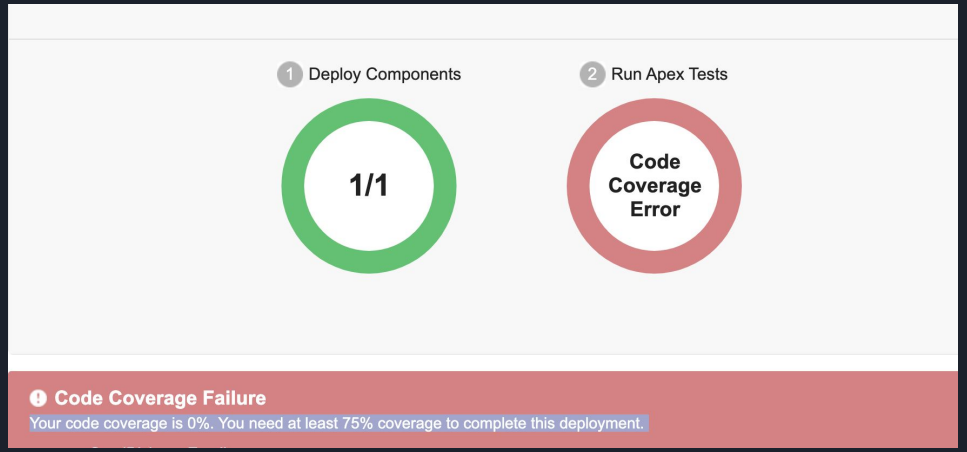
Target Audience: Developers with an intermediate level of experience who want to improve their test class writing skills.

What are test classes?

More than just a deployment requirement

Although **Salesforce** requires at least 75% of the code to be covered by test classes in order to allow deployment, it is important to understand that the role of test classes goes far beyond simply complying with this rule.

Test classes are essential to validate whether the code truly fulfills the purpose for which it was developed, ensuring its functionality across a wide range of scenarios, from the most general to the most specific. They are the real proof that the system is robust, reliable, and prepared to handle unexpected situations.



What is its importance?

Code Coverage

Salesforce requires at least 75% code coverage for deployment to production, but achieving this coverage should not be the priority!

Maintenance and Scalability

Make the code easier to maintain over time, as well as easier to modify without breaking any existing functionality.

Behavioral Documentation

A form of documentation, serving as a reference to visualize the expected behavior in a specific scenario.

Quality and Reliability

Identify and fix errors before they reach the production environment, reducing delivery delays caused by unexpected bugs.

Best Practices

F.I.R.S.T

Mentioned in the book **Clean Code** by **Robert C. Martin**, the acronym F.I.R.S.T. represents a set of principles for developing high-quality tests, promoting greater reliability, scalability, usability, and easier future maintenance.

Fast

Tests should be fast to ensure quick feedback and an efficient development cycle.

Independent

Each test should run independently, without relying on others. This prevents cascading failures and makes it easier to identify the root cause of issues.

Repeatable

Tests should be consistent, regardless of the environment or the order in which they are executed.

Self-Validating

Each test should clearly indicate success or failure on its own, without the need for manual inspection of the results.

Timely

Tests should be written in a timely manner, ideally shortly before the development of the code that will be tested.

Best Practices

Single Responsibility

Each test method should validate only a single end-to-end scenario. However, if the code allows multiple scenarios to occur within a single execution, that combination of scenarios should be considered a single test scenario.

Real Scenarios

Write tests that reflect real-world situations, using data that is as close as possible to what will be used in production. This way, your code will be better prepared to meet real-world demands.

Assertive Asserts

It is not enough to simply verify that the code runs without errors; it is essential to ensure that it returns exactly what it is supposed to return. To achieve this, use as many asserts as necessary to validate the expected results across different scenarios.

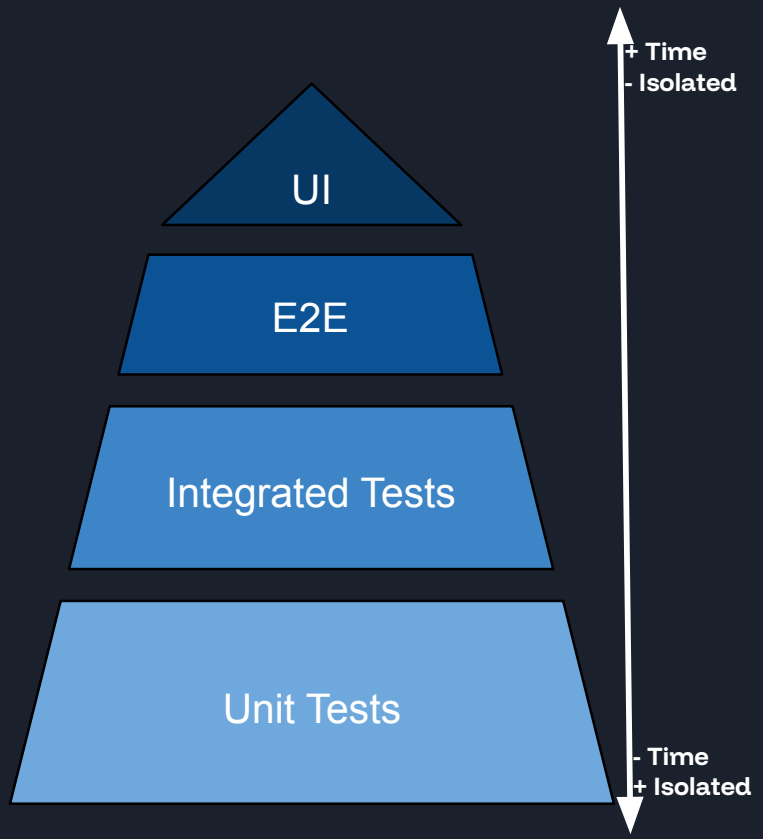
Integrated Tests vs Unit Tests

Types of Tests

A test class is not just “a test class.” Today, there are different ways to write tests, and each one is more effective within a specific context.

Since this presentation focuses specifically on Apex test classes, we will concentrate on only two types: integrated tests and unit tests.

First, we will cover the differences between these two types and when to use each one. Then, we will see how to build an almost perfect test class for each approach.



Integrated Tests vs Unit Tests

Integrated Tests

- It verifies how different components or modules of a software application work together, preventing potential bugs when combining different parts of the system.
- In this type of test, we use “real” data, performing operations such as insert, update, delete, or query directly in the database. This execution triggers triggers, validation rules, flows, and other processes associated with the object being used in the test.
- Examples of use: TriggerHandler, Controller, Repository, Apex invoked by a Flow, etc.

Unit Test

- They verify individual units of code, such as a class or method, ensuring that they function correctly across a wide variety of scenarios.
- In this type of test, there is no interaction with the database. It is performed in isolation, focusing exclusively on the class or method being tested, ensuring that its behavior is validated without external dependencies.
- Examples of use: Helper, Service, Enricher, Filter, TO/DTO, etc.

Integrated Tests vs Unit Tests

Integrated Tests

Unit Test

Just the Beginning…

In this document, I have presented only the beginning of something bigger that I am building.

So far, we have explored the theoretical foundation of how to structure an almost perfect Apex test class—understanding its importance, best practices, and different types.

But theory alone does not turn code into excellence.

In the next part of this document, we move to practice: exercises, real examples, and code (a lot of code).

If you have read this far and the content sparked your curiosity, I look forward to seeing you in part two.

Let’s evolve this together. 🚀